A FREE FIELD GUIDE

The Token *Cut* Guide

How to audit, find, and fix what your chats, bots, agents, loops, and scheduled jobs really cost to run

1. Audit
Measure where usage goes

2. Find
Spot the leaks

3. Condense
Fix, then re-measure

FREE KIT

Allen Y. Wu, JD

ETHICAL AI · FRISCO, TX

INSIDE THIS GUIDE

Contents

The principle: save by design, not by silence 3
What this guide is, and what it is not

PART 1 · AUDIT

Measure where usage really goes 4
Inventory, the cost formula, the baseline

The audit questions 5
What to ask about every consumer

Audit worksheet 6
Blank, fill-in table and baseline box

PART 2 · FIND THE PROBLEM

The ten common leaks 7
How to spot each one, and the one-line fix

Leak finder checklist and severity triage 9
Fix the top three first

PART 3 · CONDENSE AND OPTIMIZE

The fix playbook 10
Nine moves that cut waste without cutting work

Pre-flight cost check 12
One-page template for expensive work

The standing law and the result-only report 13
Paste-ready text for your own bots

The weekly 10-minute review 14
Keep the savings from drifting back

Before and after: a fictional worked example 15
Five bots and jobs, one week

Mistakes to avoid, and your next step 16
Over-correcting versus doing it right; your first hour

START HERE

Save by design, not by silence

Every chat, bot, agent, loop, and scheduled job draws on the same limited pool of usage. When that pool runs dry early, the cause is rarely one big task. It is usually many small habits repeated all day.

The principle

Token saving is a planning and design discipline. It is not silence, and it is not refusing work.

Smarter orchestration beats literal compliance. Doing it right once beats redoing it. The goal is the same work, or more, delivered with less waste.

> **The failure mode to avoid:** "never do anything that uses tokens." A bot that stops doing useful work, hides problems, or refuses tasks to save usage has not been optimized. It has been broken. Cut waste, not value.

The method in four words

Measure — Baseline before you change anything → **Find** — Name the leaks and rank them → **Fix** — Top three first, one change at a time → **Re-measure** — Compare against the baseline

How to use this guide

- Set aside one focused hour for the first pass. After that, the weekly review takes about ten minutes.
- Print the worksheets, or copy the tables into your own spreadsheet.
- Works with any AI app, automation tool, or scheduler. No special software needed.
- Results vary with your setup. The numbers you get from your own audit are the only ones that count.

PART 1 · AUDIT

Measure where usage really goes

You cannot cut what you have not counted. The audit turns a vague feeling of "we burn too much" into a short ranked list.

1. **Inventory every consumer.** List each bot, each scheduled or recurring job, each background helper, and each multi-turn loop. Include the ones you set up months ago and forgot.
2. **Record three facts for each.** How often it runs, roughly what one run costs, and which model or effort level it uses.
3. **Multiply.** Use the formula below to get a weekly total for each consumer.
4. **Find the biggest consumers.** Sort by weekly total. Then check usage by day and by bot in your AI app's usage or billing view, if it has one.
5. **Capture the usage window.** Write down what your plan allows, how much is used so far, and the exact reset date.
6. **Baseline before changing anything.** Save today's numbers. Without a baseline, you cannot tell whether a fix worked.

THE COST FORMULA

Runs per week × Cost per run × Model / effort weight

Use whatever unit your AI app reports: tokens, credits, requests, or dollars. If it shows none, use a simple 1 to 5 scale and stay consistent.

Quick runs-per-week math

Every 15 min, all day = 672

Hourly, all day = 168

Hourly, 10 working hours, weekdays = 50

Once a day = 7 · Weekdays only = 5

Estimating cost per run

Run the job once by hand and note the usage before and after. Or read recent run history. An honest estimate beats a perfect number you never get.

PART 1 · AUDIT

The audit questions

Ask these about every consumer on your list. Write the answers in the worksheet's Notes column.

Frequency

- How often does it run? Does it need to?
- Does it run nights and weekends when nobody reads the result?
- Would hourly, daily, or weekdays-only be just as useful?

Context

- How long is the chat or thread it carries?
- Are huge files, logs, or outputs pasted in every time?
- Does it re-read the same instructions on every run?

Output

- How often does a run find nothing new?
- Does it still send a message when nothing changed?
- Who reads the output, and what do they do with it?

Rework

- How often is the work redone because the request was vague?
- Is there a quality check before the result goes out?
- Does it re-check things already confirmed?

Model and effort

- Which model or effort level does it use?
- Does the task need judgment, or is it routine sorting, copying, or checking?

Ownership

- Who owns this job? Is it still needed at all?
- Does anyone look at its cost before starting big work?

The one question that matters most

"If this job stopped tomorrow, who would notice, and how soon?" If the honest answer is "nobody" or "next week", it is a candidate to slow down, merge, or retire.

PART 1 · AUDIT

Audit worksheet

One row per consumer. Weekly total = runs per week × cost per run × weight. Keep this page as your baseline.

#	Consumer (bot, job, loop)	Schedule	Runs / week	Cost / run	Model / effort	Weekly total	Notes
1							
2							
3							
4							
5							
6							
7							
8							
9							
10							

Baseline snapshot

DATE OF AUDIT

USAGE WINDOW AND RESET DATE

ALLOWED THIS WINDOW

USED SO FAR

TOP 3 DAYS BY USAGE

TOP 3 BOTS OR JOBS BY USAGE

PART 2 · FIND THE PROBLEM

The ten common leaks

Most waste falls into a few patterns. Read each one against your worksheet.

A Running too often, or around the clock

SPOT IT A job runs every few minutes, nights and weekends included, but its results only matter in working hours.

FIX **Hourly or slower, working hours only, weekdays only where it fits.**

B Finds nothing, still messages you

SPOT IT Your feed fills with "no new items" or "all clear" reports.

FIX **Stay silent when nothing changed; speak only when there is news.**

C Heavy model on light work

SPOT IT Routine sorting, copying, formatting, or checking runs on the most capable model or highest effort setting.

FIX **Match model and effort to the judgment the task really needs.**

D Redos from vague specs and skipped checks

SPOT IT The same task is run two, three, or four times because the request was unclear or errors were caught late.

FIX **Write a clear spec and add a quality check before anything ships.**

E One edit per turn

SPOT IT A long back-and-forth where each turn fixes one small thing, and each turn reloads the whole context.

FIX **Collect every edit first, then send them as one batch.**

The ten common leaks, continued

F Re-verifying what is already verified

SPOT IT The agent re-opens, re-tests, or re-reads things it confirmed earlier, with nothing changed since.

FIX **Record what is verified, and re-check only what changed.**

G Chatter: duplicates, recaps, narration

SPOT IT Duplicate or stray messages, long restatements of the request, step-by-step narration, and recaps nobody asked for.

FIX **One message per result, in a compact, result-only format.**

H Re-asking for approvals already given

SPOT IT "Should I proceed?" for things you approved last week, or a standing rule the agent keeps forgetting.

FIX **Keep one source of truth for standing rules and approvals.**

I Giant context carried along

SPOT IT Very long chats, huge pasted logs or files, full reports where a summary would do.

FIX **Start fresh threads, pass summaries, and paste only the lines that matter.**

J Nobody checks the cost before starting

SPOT IT Big jobs start without anyone asking how much they will use, and the surprise shows up at the end of the window.

FIX **Run a one-line pre-flight cost check before expensive work.**

Pattern to notice: leaks A, B, and C multiply quietly in the background. Leaks D through J show up in live conversations. Check both kinds.

PART 2 · FIND THE PROBLEM

Leak finder and triage

Leak finder checklist

- [ ] A job runs more often than hourly
- [ ] A job runs nights or weekends with no reader
- [ ] A monitor reports when nothing changed
- [ ] Routine work uses the heaviest model or effort
- [ ] A task was redone two or more times
- [ ] Edits are sent one per turn
- [ ] Confirmed items are checked again
- [ ] Messages repeat, recap, or narrate
- [ ] The bot asks for approval already given
- [ ] Chats are very long or full of pasted output
- [ ] Big work starts with no cost check

Score each ticked leak

Give each leak 1 to 3 points on three scales, then add them up.

Scale	1	3
Size	Small share of usage	Large share
Frequency	Weekly or rarer	Many times a day
Ease of fix	Needs a rebuild	A setting or a sentence

Fix the top three first. Highest score wins. Change one thing at a time so you know what worked.

Triage table

Rank	Leak and where it lives	Size	Frequency	Ease	Total	Planned fix
1						
2						
3						
4						
5						

PART 3 · CONDENSE AND OPTIMIZE

The fix playbook

Nine moves. Each one cuts waste while keeping the work.

1 Throttle schedules

Run hourly or slower. Limit to working hours. Use weekdays only where the job fits. Ask "how stale can this be before it hurts?" and set the schedule to match.

2 Silent when nothing changed

Monitors check quietly and message only when something new, broken, or urgent turns up. Real problems still get reported every time.

3 Right-size model and effort

Use lighter settings for sorting, formatting, copying, and simple checks. Save the heavy settings for judgment calls, writing that matters, and tricky problems.

4 Plan first, batch edits, execute once

"FINISH ALL THE WORK AND DECISIONS FIRST, THEN EXECUTE ONCE." Gather every requirement, decision, and edit before the first run. One complete pass costs less than five partial ones.

5 Pre-flight check for expensive work

Before a big task, flag it in one line: *"This will burn a lot of tokens."* Then offer three choices: plan it better, batch all edits first, or finish decisions first. The work still happens; it just happens once.

The fix playbook, continued

6 Quality gates that prevent redos

Before anything is delivered, check it against the spec: names, numbers, links, layout, banned words. Catching an error before delivery costs one check. Catching it after costs a full redo.

7 One source of truth for standing rules

Keep standing rules, approvals, and preferences in one place your bot reads every time, such as its instructions or a saved notes file. Then it does not need to ask again.

8 Compact message format

Lead with the result. Skip restating the request, narrating steps, and closing recaps. One message per result, no duplicates.

9 Result-only reporting

Use a fixed short template (see page 13), so every report is the same size and easy to scan.

Same task, two ways

Leaky

"Sure! You asked me to update the price list. First I opened the file, then I read every row, then I checked the totals again to be safe... Here is a recap of everything I did..."

Condensed

Done: price list updated, 12 rows changed.
Checked: totals match.
Needs you: nothing.

TEMPLATE · COPY OR PRINT

Pre-flight cost check

Use this before any task that is long, repeated, or touches many files or pages. It takes two minutes.

TASK

ESTIMATED SIZE (SMALL / MEDIUM / LARGE)

MODEL / EFFORT LEVEL

RUNS OR TURNS EXPECTED

USAGE LEFT BEFORE RESET

Ready to run? Tick all five.

- [ ] The goal and the finished result are written down in one or two sentences.
- [ ] All decisions are made; nothing will be settled halfway through.
- [ ] All edits and requirements are collected into one batch.
- [ ] The quality check is defined: what "correct" looks like.
- [ ] Only the context needed is included; no giant pastes.

If the task is large, flag it in one line

"Heads up: this will burn a lot of tokens. Want to (1) plan it better, (2) batch all edits first, or (3) finish decisions first?"

This is an offer, not a refusal. If the plan is already solid, proceed.

DECISION

☐ Proceed now ☐ Plan better first ☐ Batch edits first ☐ Finish decisions first

PASTE-READY

The standing law

Copy this paragraph into your bot's or agent's standing instructions. Edit the hours to match your own.

Token discipline (standing rule). Saving usage is a planning and design discipline, not silence and not refusing work. Finish all the work and decisions first, then execute once. Batch every edit into one pass instead of one edit per turn. Before any large task, say in one line that it will use a lot of tokens and offer to plan better, batch edits first, or finish decisions first; if the plan is already solid, proceed. Match the model and effort level to the judgment the task needs. Run scheduled jobs hourly or slower, during working hours and on weekdays unless told otherwise. When a check finds nothing new, stay silent; always report real problems. Do not re-verify what is already verified unless something changed. Do not ask again for approvals already given; read the standing rules first. Check work against the spec before delivering, so it is done right once. Report results only: no restating the request, no narration, no recaps, no duplicate messages.

Result-only report template

Done: what was finished, in one line

Result: the file, link, or answer

Checked: what was verified before delivery

Unsure: anything that needs a second look, or "none"

Needs you: a decision or approval, or "nothing"

Keep the safety valve: the "Unsure" and "Needs you" lines stay even when short. Compact reporting should never hide a problem.

ROUTINE

The weekly 10-minute review

Savings drift back when nobody looks. Pick one day and time each week, and keep it short.

Min 1–2 **Read the numbers.** Usage so far, days to reset, and the top three bots or jobs and top three days. Compare with last week and with your baseline.

Min 3–4 **Scan for new consumers.** Any new bot, scheduled job, or loop added this week? Add it to the worksheet with its schedule and model level.

Min 5–6 **Run the leak finder.** Tick the checklist on page 9 against anything that rose. One new leak a week is normal; catch it early.

Min 7–8 **Pick one fix.** The highest-scoring leak only. Write down what you will change and when.

Min 9–10 **Check last week's fix.** Did usage drop for that job? Did the work still get done? If quality slipped, roll it back and try a gentler change.

Weekly log

Week of	Used vs. last week	Fix made	Result after re-measure

WORKED EXAMPLE

Before and after

> **FICTIONAL** **Made-up business, made-up numbers.** "Maple Lane Studio" is invented to show the math. Units are imaginary usage units. Your results will differ.

Bot or job	Before	Units / wk	After	Units / wk
Inbox monitor	Every 15 min, all week, heavy model (672 runs × 2), posts "nothing new"	1,344	Hourly 8–6, weekdays, light model (55 × 1), silent when empty	55
Price checker	Hourly, all week (168 × 3), re-checks the same items	504	Twice a day, weekdays (10 × 3), checks only changed items	30
Social post drafter	10 posts, about 3 attempts each from vague briefs (30 × 15)	450	Clear brief + quality gate; 10 first drafts plus 2 redos (12 × 15)	180
Website edit helper	One edit per turn (30 turns × 8)	240	Edits batched into 5 turns (5 × 12)	60
Daily sales summary	Every day, full data pasted, long recap (7 × 20)	140	Weekdays, trimmed context, result-only (5 × 12)	60
Fictional weekly total		**2,678**		**385**

What the example shows

- The two background jobs were the largest consumers, and nobody had looked at them since setup.
- Every job still runs and every post still ships. The work stayed; the waste went.
- The fixes were mostly settings and sentences: a schedule, a silent rule, a model level, a batching habit.
- Real savings depend on your own setup. Measure, find, fix, re-measure.

BEFORE YOU GO

Mistakes to avoid

Over-correcting

- "Never do anything that uses tokens"
- Silencing alerts that report real problems
- Dropping quality checks to save a run
- Refusing or stalling work that matters
- Changing five things at once, then guessing

Doing it right

- Baseline first, then change one thing
- Silent only when nothing changed
- Gates before delivery, so no redos
- Plan, batch, then execute once
- Re-measure and keep what worked

Your first hour

1. Fill in the audit worksheet and baseline snapshot (page 6).
2. Tick the leak finder and score your leaks (page 9).
3. Apply the fix for your top three, one at a time (pages 10–11).
4. Paste the standing law into your bots' instructions (page 13).
5. Put the weekly 10-minute review on your calendar (page 14).

Doing it right once beats redoing it.

Allen Y. Wu, JD
ETHICAL AI · FRISCO, TX

More free kits and guides at **ethicalai.me**

This guide is general educational information about managing AI usage. It is not legal, financial, or professional advice.